Blockchain : Experiment - 9

Aim:

Creating a Token (ERC-20) in Remix IDE.

Theory:

ERC-20 (Ethereum Request for Comments 20) is a technical standard used for creating and implementing fungible tokens on the Ethereum blockchain. Fungible tokens are digital assets in which each token is identical and interchangeable with another token of the same type.
ERC-20 defines a common set of functions and events that allow tokens to interact with wallets, decentralized applications (DApps), and cryptocurrency exchanges.

1. **Smart Contracts:** A smart contract is a self-executing program deployed on a blockchain. It automatically executes predefined instructions when specific conditions are met. ERC-20 tokens use smart contracts to manage token creation, transfers, balances, and approvals.
2. **Solidity:** Solidity is a high-level, object-oriented programming language used to develop smart contracts on Ethereum-compatible blockchains. It is used to define the functionality and rules of ERC-20 tokens.
3. **Remix IDE:** Remix IDE is a browser-based development environment used to write, compile, deploy, and debug Solidity smart contracts. It provides tools such as File Explorer, Solidity Compiler, and Deploy & Run Transactions.
4. **ERC-20 Token Functions:** The ERC-20 standard defines the following important functions:
    a. totalSupply(): Returns the total number of tokens created.
    b. balanceOf(address): Returns the token balance of a particular account.
    c. transfer(address, amount): Transfers tokens from one account to another.
    d. approve(address, amount): Allows another account to spend a specified number of tokens.
    e. allowance(owner, spender): Returns the remaining approved token amount.
    f. transferFrom(from, to, amount): Transfers tokens on behalf of another account using an approved allowance.
5. **ERC-20 Events:** Events are used to record important activities performed by a smart contract.
    a. Transfer: Emitted whenever tokens are transferred between accounts, including token minting and burning.
    b. Approval: Emitted whenever an account approves another account to spend tokens.
6. **MetaMask Wallet:** MetaMask is a cryptocurrency wallet used to manage blockchain accounts and interact with decentralized applications. It allows users to connect Remix IDE to an Ethereum network, approve transactions, and pay gas fees.
7. **Sepolia Testnet:** Sepolia is an Ethereum test network used for developing and testing smart contracts without spending real cryptocurrency. Developers use test ETH to deploy contracts and execute transactions.
8. **Token Deployment:** After writing and compiling the Solidity program, the smart contract is deployed to the blockchain using Remix IDE and MetaMask. Once deployed, the contract receives a unique address and its functions can be accessed through Remix or other blockchain applications.
9. **OpenZeppelin:** OpenZeppelin provides reusable and standardized Solidity libraries for developing secure smart contracts. Its ERC-20 implementation simplifies token creation by providing predefined functions for managing balances, transfers, approvals, and token supply.
10. **Etherscan:** Etherscan is a blockchain explorer used to inspect transactions, smart contract addresses, blocks, and token activities. Sepolia Etherscan allows developers to verify their deployed smart contracts on the Sepolia Testnet.

Code :

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "@openzeppelin/contracts/token/ERC20/ERC20.sol";

contract ShaktiToken is ERC20 {

   constructor() ERC20("Shakti Token", "SHK") {
      _mint(msg.sender, 1000000 * 10 ** decimals());
   }
}
```

Output :

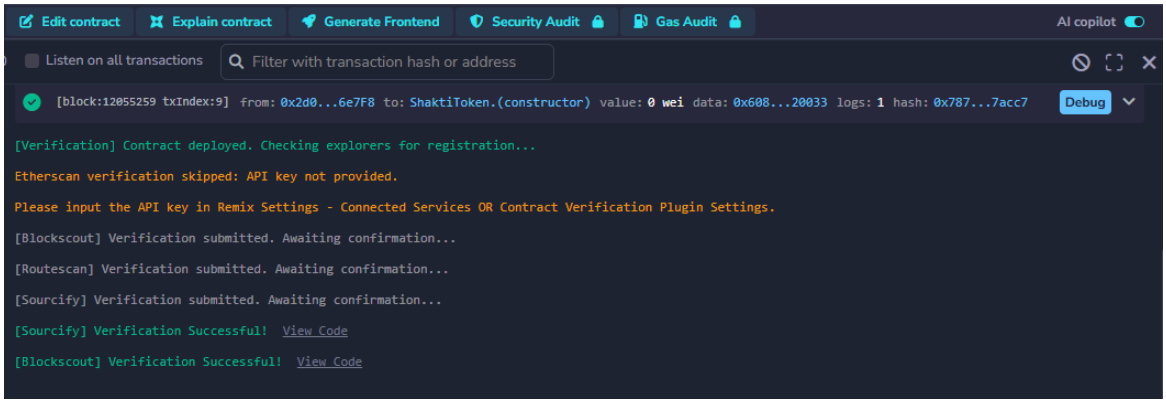
Complete all 6 LearnETH course chapters:

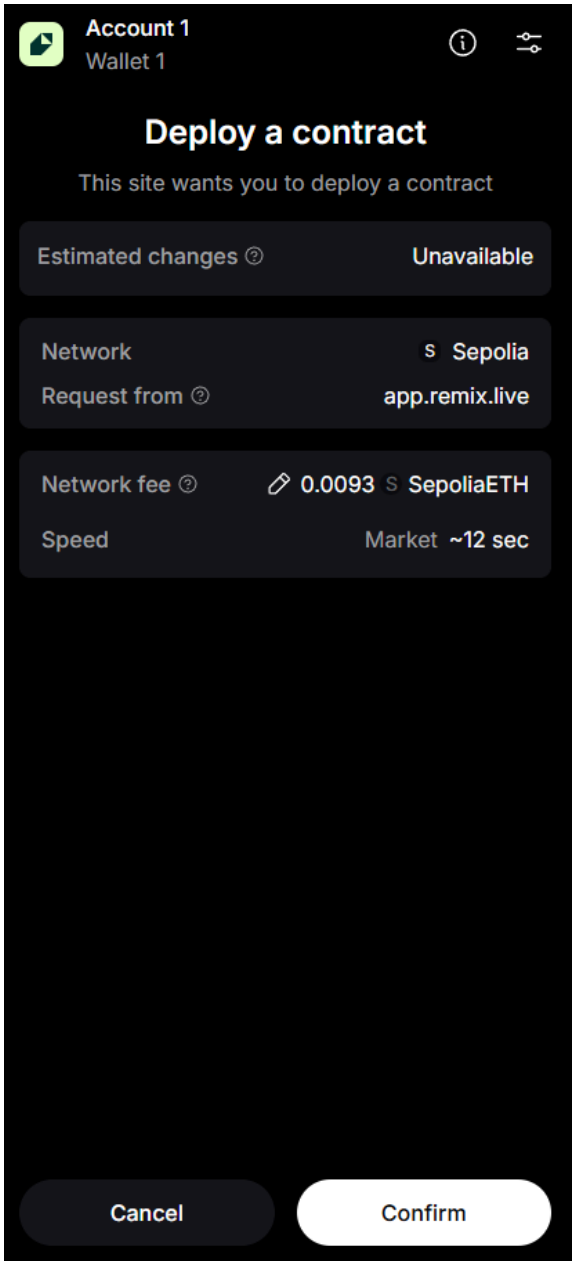


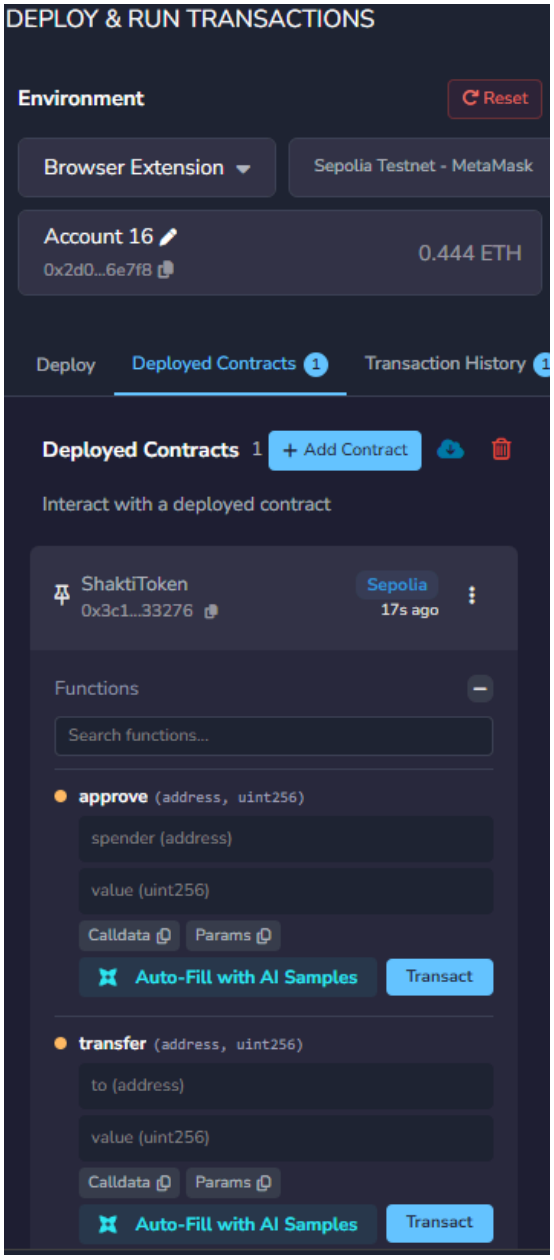
Create ERC-20 Solidity program and then compile it:

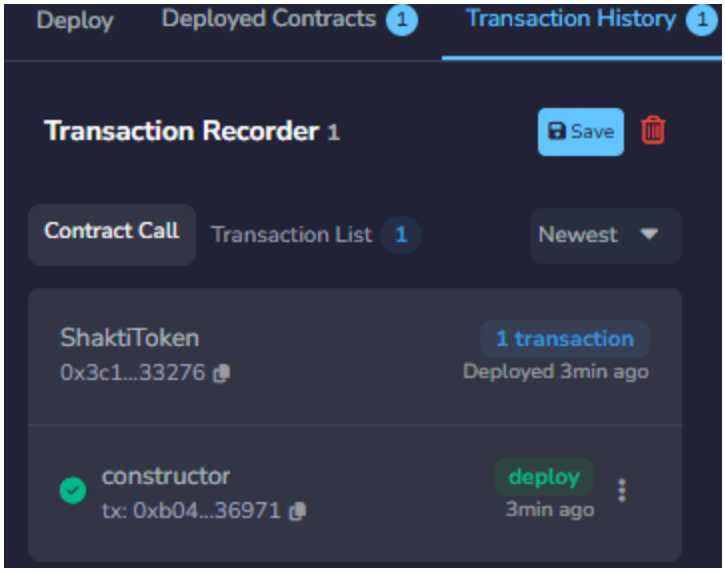


Deploy and Run the smart contract:

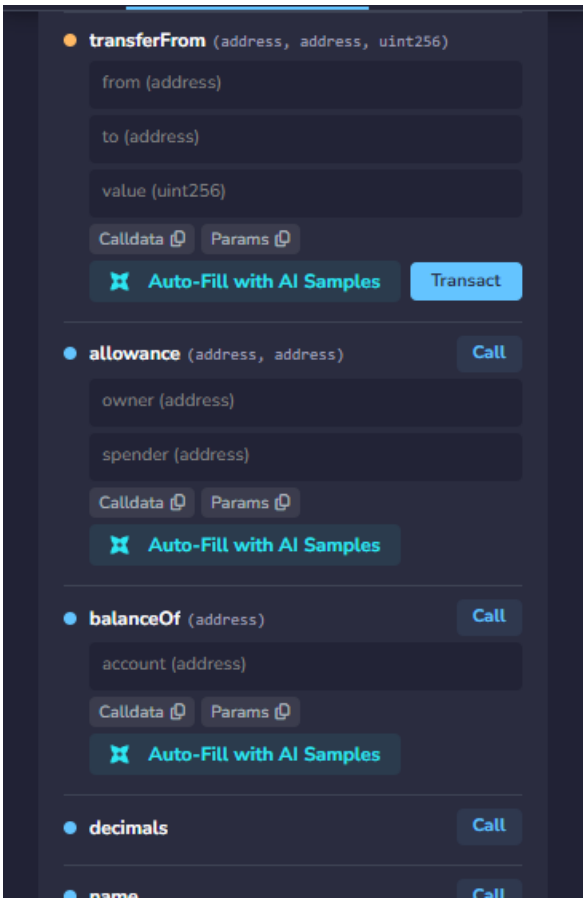


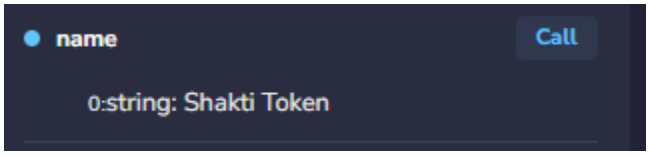




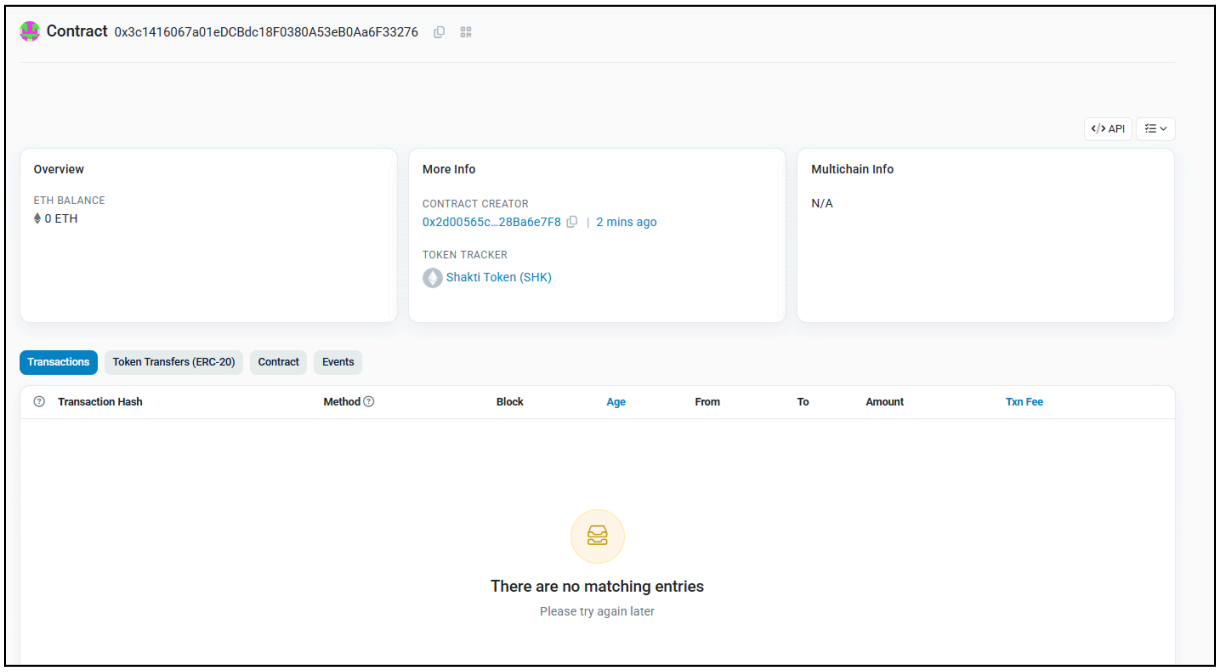


Functions available after deployment:





Smart Contract on Etherscan:



Conclusion:

Successfully studied the ERC-20 token standard and created an ERC-20 token using Solidity in Remix IDE. Configured MetaMask, connected it to the Sepolia Testnet, and understood the compilation, deployment, and execution of smart contract functions. Also understood ERC-20 token functions, events, and the verification of deployed smart contracts using Etherscan.